

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №4.2
по дисциплине
«Программирование»

Выполнил студент гр. ИВТб-1303-06-00

_____/Гортоломей И.К./

Проверил преподаватель кафедры ЭВМ

_____/Баташев П.А./

Киров

2026

Цель работы:

Научиться создавать простые графические интерфейсы с использованием библиотеки tkinter, организовывать обработку событий и интегрировать ранее разработанные программные модули в единое приложение. Закрепить понимание работы выбранной структуры данных через её визуальное представление.

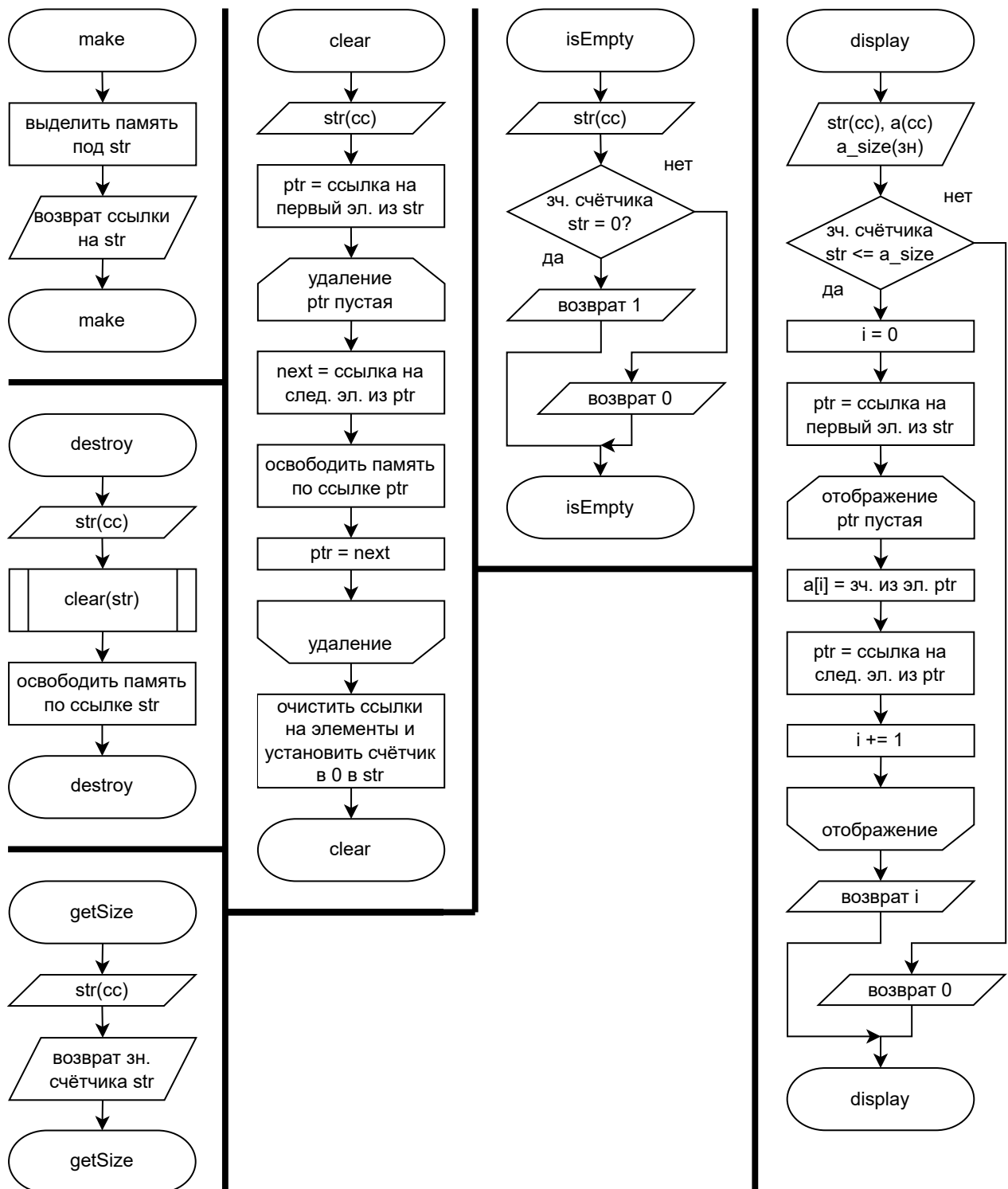
Реализуемая структура:

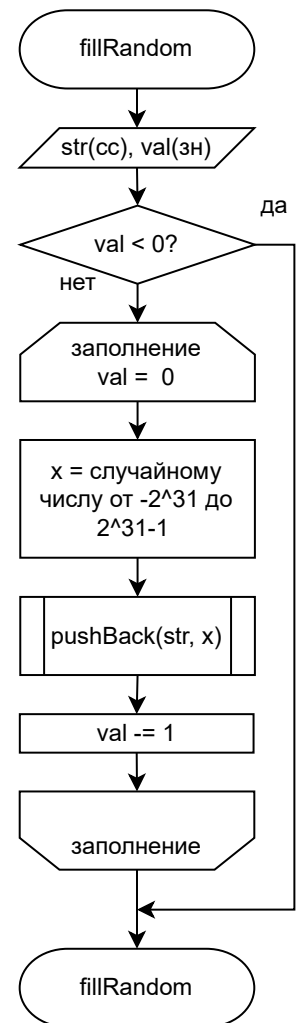
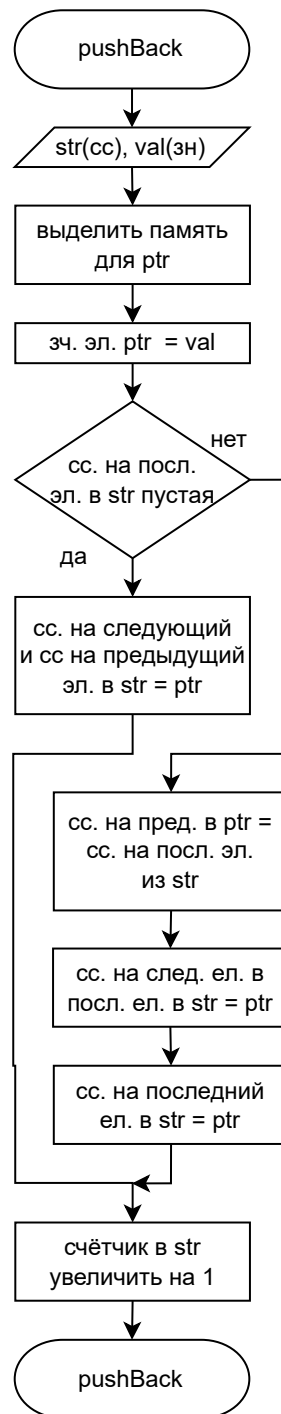
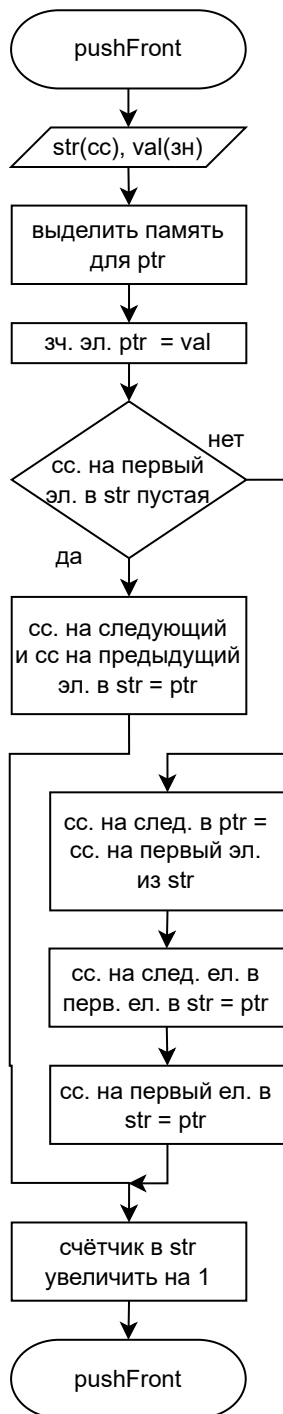
Дек (deque) с функцией вставки случайных чисел.

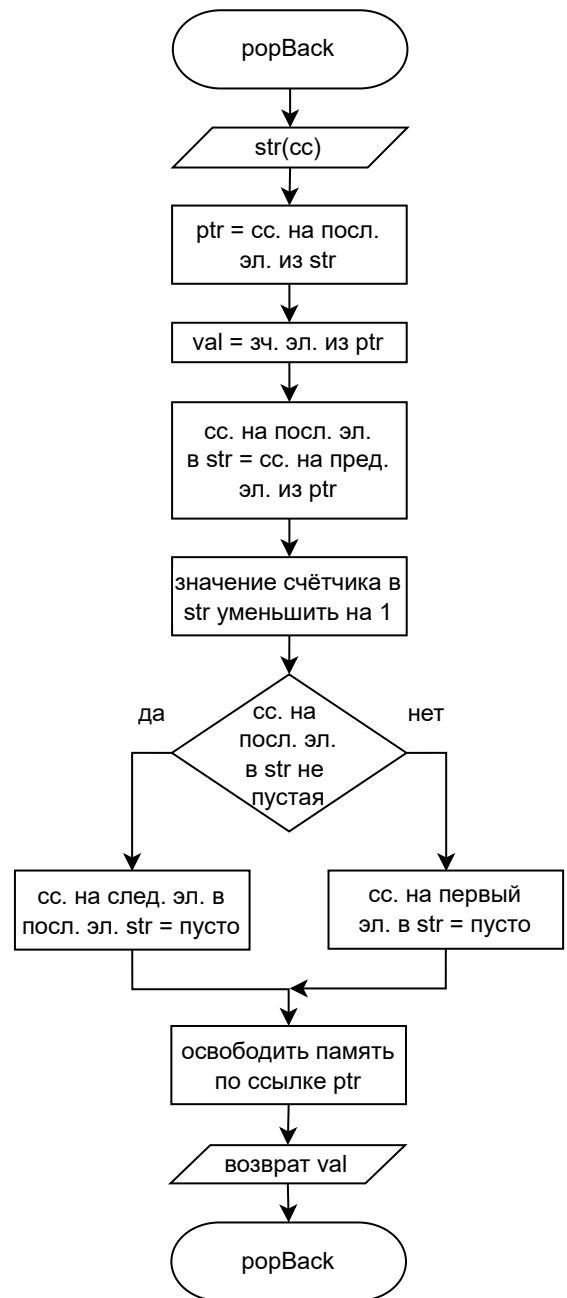
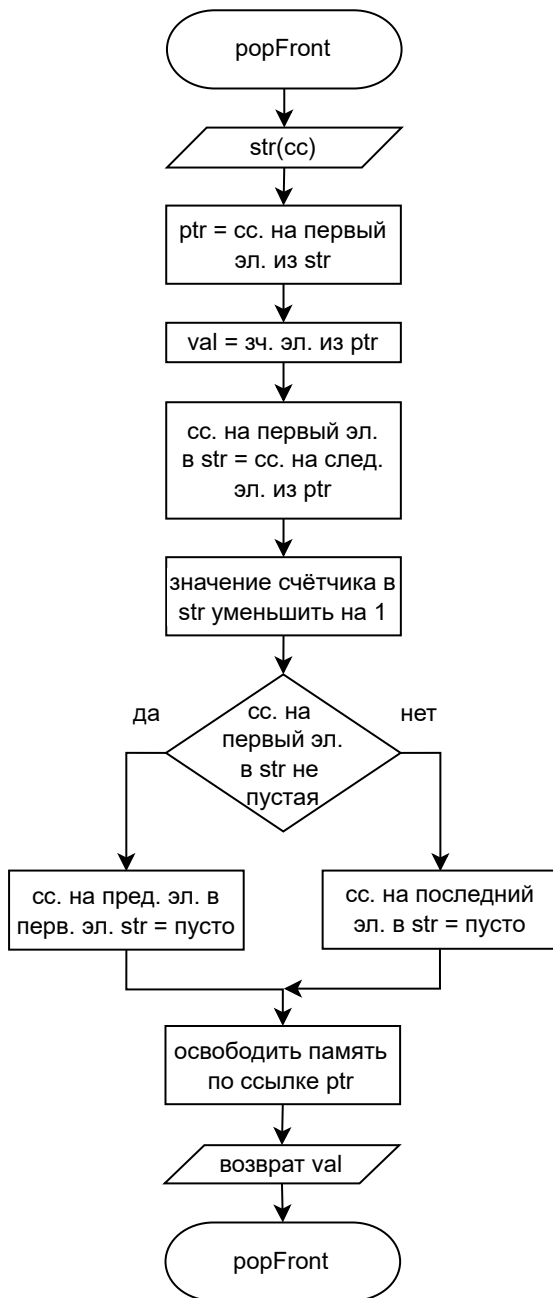
Среда разработки:

Visual Studio Code на Windows 11 и компилятор в MSYS2 UCRT64.

Схемы алгоритмов функций







Заголовочный файл для модулей на C++

```
1 #pragma once
2 #define Export __declspec(dllexport)
3
4 extern "C" {
5     Export void* make();
6     Export void clear(void* handle);
7     Export void destroy(void* handle);
8
9     Export int getSize(void* handle);
10    Export bool isEmpty(void* handle);
11
12    Export void pushFront(void* handle, int value);
13    Export void pushBack(void* handle, int value);
14    Export void fillRandom(void* handle, int value);
15
16    Export int popFront(void* handle);
17    Export int popBack(void* handle);
18
19    Export int display(void* handle, int* buffer, int buffer_size);
20 }
```

Модуль на C++

```
1 #include "mod.h"
2 #include <time.h>
3 #include <stdlib.h>
4
5 bool initRandom = false;
6
7 struct Element {
8     int data;
9     Element *prev;
10    Element *next;
11 };
```

```

12
13 Export void* make() {
14     if (!initRandom) {
15         srand(time(0));
16         initRandom = true;
17     }
18     Element* ptr = new Element;
19     ptr->data = 0;
20     ptr->prev = nullptr;
21     ptr->next = nullptr;
22     return ptr;
23 }
24
25 Export void clear(void* handle) {
26     Element* str = (Element*) handle;
27     Element* ptr = str->prev;
28     while (ptr != nullptr) {
29         Element* next = ptr->next;
30         delete ptr;
31         ptr = next;
32     }
33     str->prev = nullptr;
34     str->next = nullptr;
35     str->data = 0;
36 }
37
38 Export void destroy(void* handle) {
39     Element* str = (Element*) handle;
40     clear(str);
41     delete str;
42 }
43
44 Export int getSize(void* handle) {
45     Element* str = (Element*) handle;
46     return str->data;
47 }

```

```

48
49 Export bool isEmpty(void* handle) {
50     Element* str = (Element*) handle;
51     return str->data == 0;
52 }
53
54 Export void pushFront(void* handle, int value) {
55     Element* str = (Element*) handle;
56     Element* ptr = new Element;
57     ptr->data = value;
58     ptr->prev = nullptr;
59     ptr->next = nullptr;
60     if (str->prev == nullptr) {
61         str->prev = ptr;
62         str->next = ptr;
63     } else {
64         ptr->next = str->prev;
65         str->prev->prev = ptr;
66         str->prev = ptr;
67     }
68     str->data++;
69 }
70
71 Export void pushBack(void* handle, int value) {
72     Element* str = (Element*) handle;
73     Element* ptr = new Element;
74     ptr->data = value;
75     ptr->prev = nullptr;
76     ptr->next = nullptr;
77     if (str->next == nullptr) {
78         str->prev = ptr;
79         str->next = ptr;
80     } else {
81         ptr->prev = str->next;
82         str->next->next = ptr;
83         str->next = ptr;

```



```

84     }
85     str->data++;
86 }
87
88 Export void fillRandom(void* handle, int value) {
89     Element* str = (Element*) handle;
90     if (value < 0) return;
91     for (; value > 0; value--) {
92         unsigned int x = (rand() << 17) + (rand() << 2) + (rand() & 3);
93         pushBack(handle, (signed int) x);
94     }
95 }
96
97 Export int popFront(void* handle) {
98     Element* str = (Element*) handle;
99     Element* ptr = str->prev;
100    int value = ptr->data;
101    str->prev = ptr->next;
102    str->data--;
103    if (str->prev != nullptr) {
104        str->prev->prev = nullptr;
105    } else {
106        str->next = nullptr;
107    }
108    delete ptr;
109    return value;
110 }
111
112 Export int popBack(void* handle) {
113     Element* str = (Element*) handle;
114     Element* ptr = str->next;
115     int value = ptr->data;
116     str->next = ptr->prev;
117     str->data--;
118     if (str->next != nullptr) {
119         str->next->next = nullptr;

```

```

120     } else {
121         str->prev = nullptr;
122     }
123     delete ptr;
124     return value;
125 }
126
127 Export int display(void* handle, int* buffer, int buffer_size) {
128     Element* str = (Element*) handle;
129     if (buffer && str->data <= buffer_size) {
130         int count = 0;
131         Element* ptr = str->prev;
132         while (ptr != nullptr) {
133             buffer[count] = ptr->data;
134             ptr = ptr->next;
135             count++;
136         }; return count;
137     }; return 0;
138 }

```

Модуль на Python

```

1  from random import randint
2
3
4  class Element:
5      def __init__(self):
6          self.data: int = 0
7          self.prev: Element = None
8          self.next: Element = None
9
10 def make() -> Element:
11     return Element()
12
13 def clear(str: Element) -> None:
14     ptr = str.prev;

```

```

15     while (ptr != None):
16         next = ptr.next;
17         del ptr
18         ptr = next
19     str.prev = None
20     str.next = None
21     str.data = 0
22
23 def destroy(str: Element) -> None:
24     clear(str)
25     del str
26
27 def getSize(str: Element) -> int:
28     return str.data
29
30 def isEmpty(str: Element) -> bool:
31     return str.data == 0
32
33 def pushFront(str: Element, value: int) -> None:
34     ptr = Element()
35     ptr.data = value
36     ptr.prev = None
37     ptr.next = None
38     if str.prev == None:
39         str.prev = ptr
40         str.next = ptr
41     else:
42         ptr.next = str.prev
43         str.prev.prev = ptr
44         str.prev = ptr
45     str.data += 1
46
47 def pushBack(str: Element, value: int) -> None:
48     ptr = Element()
49     ptr.data = value
50     ptr.prev = None

```

```

51 ptr.next = None
52 if str.next == None:
53     str.prev = ptr
54     str.next = ptr
55 else:
56     ptr.prev = str.next
57     str.next.next = ptr
58     str.next = ptr
59 str.data += 1
60
61 def fillRandom(str: Element, value: int) -> None:
62     if value < 0: return
63     while value > 0:
64         pushBack(str, randint(-2**31, 2**31-1))
65         value -= 1
66
67 def popFront(str: Element) -> int:
68     ptr = str.prev
69     value = ptr.data
70     str.prev = ptr.next
71     str.data -= 1
72     if str.prev != None:
73         str.prev.prev = None
74     else:
75         str.next = None
76     del ptr
77     return value
78
79 def popBack(str: Element) -> int:
80     ptr = str.next
81     value = ptr.data
82     str.next = ptr.prev
83     str.data -= 1
84     if str.next != None:
85         str.next.next = None
86     else:

```

```

87     str.prev = None
88 del ptr
89 return value
90
91 def display(str: Element, buffer: list[int], buffer_size: int) ->
    int:
92     if str.data <= buffer_size:
93         count: int = 0
94         ptr = str.prev
95         while (ptr != None):
96             buffer[count] = ptr.data
97             ptr = ptr.next
98             count += 1
99         return count
100     return 0

```

Модуль на C++ (с STL)

```

1  #include "mod.h"
2
3  #include <time.h>
4  #include <stdlib.h>
5
6  #include <deque>
7  // #include <cstdint>
8
9  bool initRandom = false;
10 #define IntDeque std::deque<int>
11 #define Cast(x) static_cast<IntDeque*>(x)
12
13 Export void* make() {
14     if (!initRandom) {
15         srand(time(0));
16         initRandom = true;
17     }
18     try {

```

```

19     return new IntDeque();
20 } catch (...) {
21     return nullptr;
22 }
23 }
24
25 Export void destroy(void* handle) {
26     if (handle == nullptr) return;
27     delete Cast(handle);
28 }
29
30 Export void clear(void* handle) {
31     if (handle == nullptr) return;
32     Cast(handle)->clear();
33 }
34
35 Export int getSize(void* handle) {
36     if (handle == nullptr) return 0;
37     return Cast(handle)->size();
38 }
39
40 Export bool isEmpty(void* handle) {
41     if (handle == nullptr) return false;
42     return Cast(handle)->empty();
43 }
44
45 Export void pushFront(void* handle, int value) {
46     if (handle == nullptr) return;
47     Cast(handle)->push_front(value);
48 }
49
50 Export void pushBack(void* handle, int value) {
51     if (handle == nullptr) return;
52     Cast(handle)->push_back(value);
53 }
54

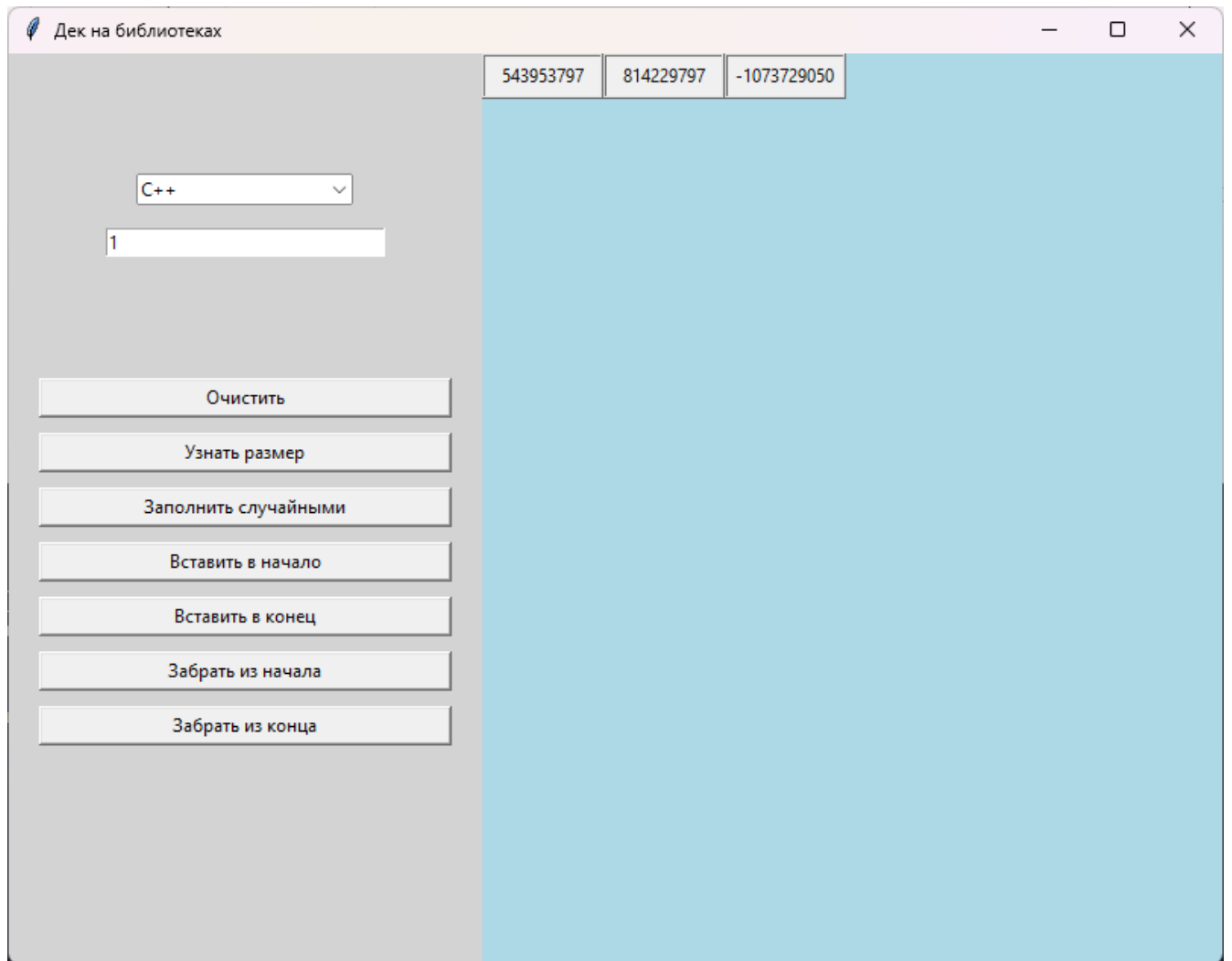
```

```

55 Export void fillRandom(void* handle, int value) {
56     if (handle == nullptr) return;
57     auto *deq = Cast(handle);
58     if (value < 0) return;
59     for (; value > 0; value--) {
60         unsigned int x = (rand() << 17) + (rand() << 2) + (rand() & 3);
61         pushBack(handle, (signed int) x);
62     }
63 }
64
65 Export int popFront(void* handle) {
66     if (handle == nullptr) return 0;
67     auto *deq = Cast(handle);
68     if (deq->empty()) return 0;
69     auto value = deq->front();
70     deq->pop_front();
71     return value;
72 }
73
74 Export int popBack(void* handle) {
75     if (handle == nullptr) return 0;
76     auto *deq = Cast(handle);
77     if (deq->empty()) return 0;
78     auto value = deq->back();
79     deq->pop_back();
80     return value;
81 }
82
83 Export int display(void* handle, int* buffer, int buffer_size) {
84     auto* d = Cast(handle);
85     int count = d->size();
86     if (buffer && count <= buffer_size) {
87         std::copy(d->begin(), d->end(), buffer);
88         return count;
89     }; return 0;
90 }

```

Скриншот Графического Пользовательского Интерфейса



В правой части (на цветном фоне) отображаются элементы структуры. А в левой части сверху вниз: выпадающий список для выбора модуля, поле для ввода/вывода числа и кнопки действий.

Выводы

В ходе выполнения работы была достигнута основная цель: освоены принципы создания графического интерфейса на tkinter и его интеграция с ранее реализованными модулями структуры данных «Дек». Анализ трёх версий библиотек (ручная реализация на C++, версия на STL и Python-модуль) показал их функциональную идентичность при сохранении единого интерфейса взаимодействия. Разработка GUI-оболочки позволила наглядно продемонстрировать работу дека: процессы добавления/удаления элементов с обоих концов, заполнения случайными числами и отображения содержимого. Визуализация подтвердила корректную работу всех трёх реализаций и помогла глубже понять логику двусторонней очереди. Таким образом, использование tkinter в качестве связующего звена между интерфейсом и вычислительными модулями доказало свою эффективность для создания учебных приложений и отладки структур данных.